APPLIED ARTIFICIAL INTELLIGENCE / TECHNICAL REPORT

RoboGuide AI

A traceable, version-aware PiDog troubleshooting assistant

Functional CPU-first prototype | Data engineering | Hybrid retrieval | Safety gating | Reproducible evaluation

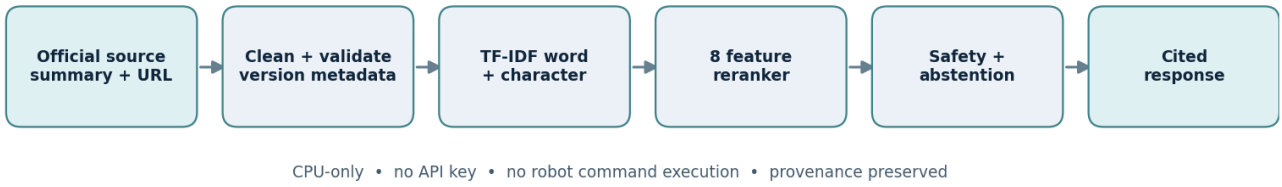


Figure 1. Actual offline prototype workflow: version-aware evidence retrieval and extractive citation generation.

COURSE	GROUP / AUTHORS
ITAI 2377 — Data Science for AI	Tung Tung Tung Sahar José Tuozzo • Kim Nguyen • Huu Huynh
REPORT STATUS	SUBMISSION CONVENTION
Reproducible technical draft; teammate statements need sign-off	FN_JoseTuozzo_TungTungTungSahar_ITAI2377.pdf
DATE	PLANNED COURSE DELIVERY
November 10, 2026	December 2, 2026 (Canvas course overview)

Integrity statement: All quantitative results in this report were generated by executing the included scripts locally. This is an educational prototype, not a deployed support service, and not a manufacturer-endorsed product.

01 Executive summary

RoboGuide transforms symptom-level PiDog questions into source-linked, actionable manufacturer guidance. The key problem is that students describe a shaky gait, silent speaker or sensor failure in everyday language while the manual is organized by components, versions and APIs. The implemented prototype indexes 24 labeled, source-attributed summaries of official SunFounder documentation and retrieves relevant evidence without a GPU, API key or robot connection.

Measured pilot results

Metric	Development (35 queries)	Fixed test slice (8 queries)
Hybrid Recall@3	87.1%	100.0%
Hybrid MRR@5	83.9%	100.0%
TF-IDF baseline MRR@5	77.4%	100.0%
Status classification accuracy	97.1%	87.5%
Median response latency	2.12 ms	2.33 ms

Development MRR improved from 0.7742 to 0.8387 (+0.0645 absolute). The fixed test slice has identical retrieval metrics for both methods (MRR@5 = 1.0), so there is no demonstrated held-out retrieval gain in these eight cases. Stronger claims require more independent queries.

What is implemented

- Auditable knowledge schema with version, subsystem, authority and direct source URLs.
- Canonical cleaning, staged data-quality comparisons and a safe online source-ingestion utility.
- Word/character TF-IDF retrieval plus eight explicit engineered ranking signals; categorical encoding and numeric normalization.
- Deterministic safety escalation, low-evidence abstention and cited extractive answers.
- 43-question benchmark, Recall@3, MRR@5, error tables, permutation feature importance, ablation experiments, 19 passing automated tests and Colab notebook.

Not demonstrated: a 150–250 independently labeled benchmark, full 80–150-source crawl, FAISS/Sentence Transformers evaluation, human citation scoring, calibrated probabilistic confidence or physical PiDog validation. Those were planned targets, not outcomes.

02 Alignment with the midterm plan and assignment

The instructor praised the original feature engineering table, particularly error fingerprinting, evidence confidence gating and a CPU-only fallback, but requested who completed the plan-writing, diagrams and references in addition to future job ownership. This report retains these strengths and includes a contribution-verification protocol.

Midterm design / final requirement	Implementation evidence	Status
Manufacturer knowledge base; data schema	24 curated official-source summaries with source_url, version and subsystem	Implemented pilot
Missing values / duplicates / text cleaning	Intentionally dirtied staging fixture, canonical restoration, before/after metrics	Demonstrated
Numerical scaling and categorical encoding	StandardScaler + OneHotEncoder fit on development candidates	Implemented
3–5 engineered features + importance	8 features, grouped-question permutation AP and MRR ablation	Implemented
Semantic embeddings and FAISS	CPU TF-IDF word + character vectors; no neural/FAISS benchmark	Adapted
Evidence-based interface and five demos	Command-line + Gradio; eight reproducible conversations	Implemented
Evaluation / comparison to baseline	43 manually authored questions; deterministic fixed split	Pilot scope
Team contributions for completed work	Signed statements still needed from each named member	Pending verification

Why the architecture changed

The plan proposed 384-dimensional MiniLM embeddings, a FAISS vector index, and an optional small instruction generator. With restricted network access and CPU-first requirements, the evaluated build instead uses scikit-learn TF-IDF and deterministic extractive synthesis. This lowers setup friction, eliminates generative hallucination from the answer-writing step, and exposes interpretable ranking signals. It also reduces semantic paraphrase capacity and must not be represented as having evaluated MiniLM, FAISS or a language model.

03 Data provenance and quality

The offline corpus has 24 manually curated, paraphrased summaries of sections in the SunFounder PiDog FAQ, calibration guide, Python API lessons, official PiDog GitHub repository and Robot HAT documentation. These are not 24 live HTML downloads. URLs, source_type, document IDs, hardware version, subsystem, review status and attribution are retained in each record.

Schema and update strategy

Field	Purpose / validation
id / title	Unique document key and concise human-readable evidence title
content	Text summary preserving important commands and model names
version	One of standard, v2, shared; hard exclude incompatible chunks
subsystem	Hardware/software category for diagnostics and analysis
source_url / source_type	HTTPS source provenance and explicit curated-summary label
authority / retrieved_on	Manufacturer-backed status and a date of documentation review
review_status	Reminder that physical steps require human cross-check

Controlled data corruption experiment

The raw staging fixture intentionally introduces two duplicate IDs, two missing required cells, one boilerplate record and one invalid version tag. The cleaning stage uses a trusted canonical key to restore those fields, strips noise and drops repeats. This verifies a repair mechanism; it is not an observation that official documentation was corrupt.

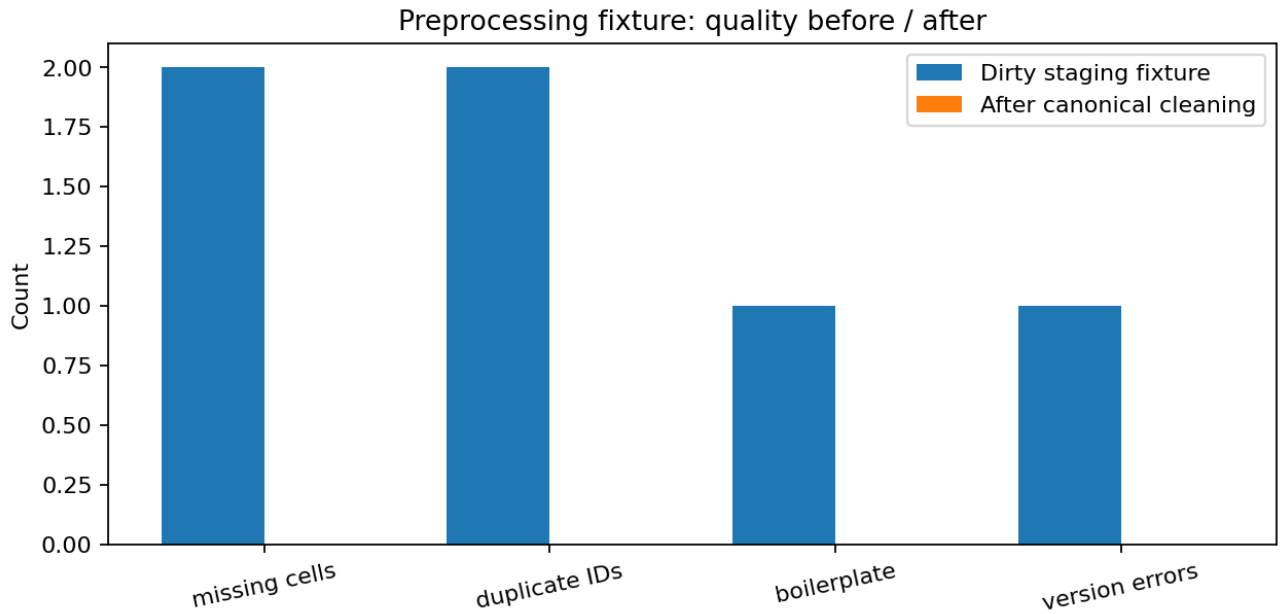


Figure 2. Explicitly simulated data-quality defects before versus after preprocessing.

04 Data transformation and engineered features

Two vectorizers are independently fitted to the canonical corpus: word unigrams/bigrams and character 3–5-grams with sublinear TF-IDF weights. Each candidate evidence record receives eight numeric features. For exploratory relevance modeling, numeric values are standardized using development-only statistics, and version/subsystem categories are encoded with a development-only one-hot vocabulary. The ranking formula itself consumes its directly interpretable raw [0,1]-scale features.

Feature	Signal	Why it exists
Word similarity	TF-IDF cosine	Lexical evidence matching; baseline
Character similarity	Character-gram cosine	Robustness to spelling and token variations
Entity overlap	PiDog terminology dictionary	Sensors, servo, RGB, API names
Title overlap	Normalized meaningful words	Boosts exact procedure matches
API match	Methods/function identifiers	Preserves code-token specificity
Error fingerprint match	Exception class + normalized message	Merges path/line-variant failures
Version compatibility	Standard / V2 / shared	Blocks incompatible hardware steps
Source authority	Explicit source score	Prioritizes manufacturer-supported evidence

Fingerprint demonstration: different stack-trace paths and line numbers with the same ModuleNotFoundError: No module named robot_hat normalize to the same stable signature. This behavior is tested; benchmark attribution to fingerprint is zero because the tiny corpus contains no matching traceback documents. A zero measured importance is not the same as an unimplemented feature.

05 Feature importance and interpretability

A candidate-level logistic relevance model was trained on ranking features for exploratory permutation analysis. Train/test candidate sets were grouped by query ID to limit leakage. The query-group holdout average precision was 0.598 over 10 questions. The plot reports change in average precision when one feature is shuffled: positive values indicate helpful signal under this specific setup; negative values indicate instability, redundancy or possible overfit, not intrinsic harm.

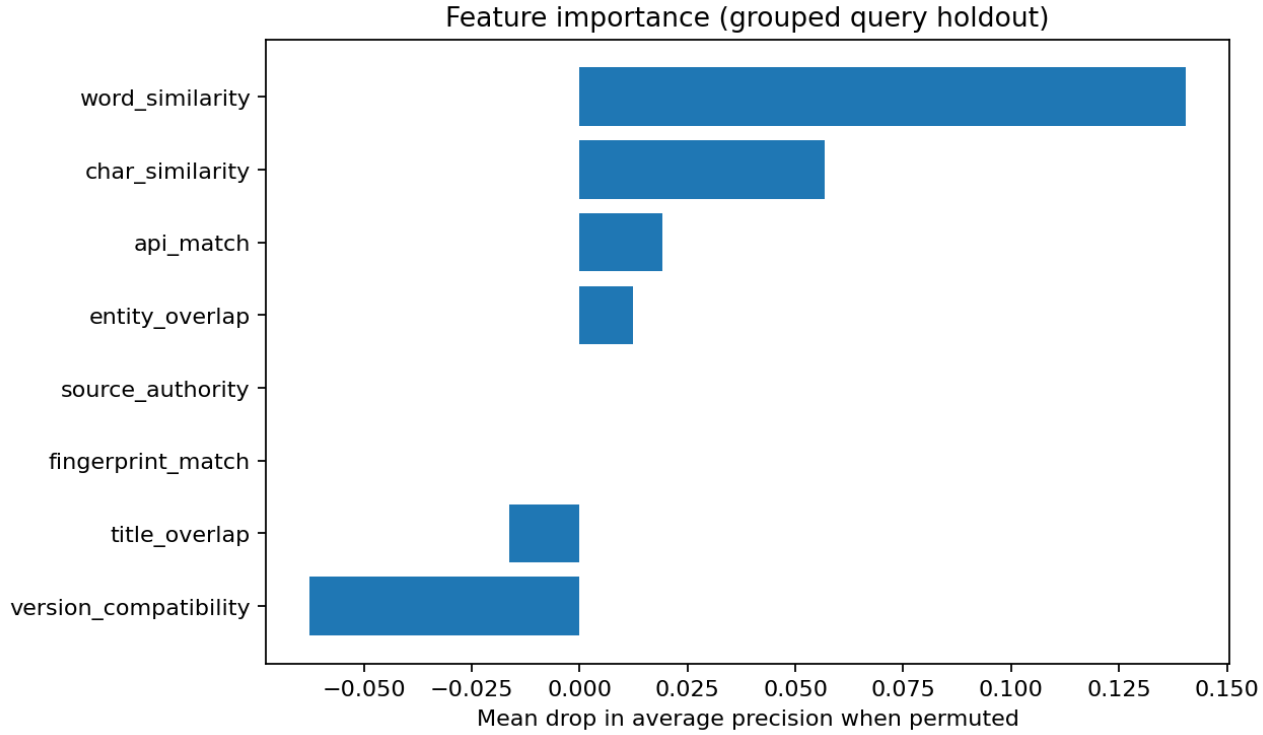


Figure 3. Grouped-query permutation importance on candidate relevance. Small-sample estimate; not a causal explanation.

Ranking-feature ablations

Feature disabled	Development MRR@5 decrease
char_similarity	+0.0500
title_overlap	+0.0419
entity_overlap	+0.0312
word_similarity	+0.0097
api_match	+0.0000
fingerprint_match	+0.0000
version_compatibility	+0.0000
source_authority	+0.0000

06 Core interaction, guardrails and UI

The system exposes `RoboGuide.search()` to retrieve scored evidence and `RoboGuide.ask()` to deliver a structured status, cited extractive response and latency. A Gradio interface and Python CLI exercise the same core service. The application never opens a serial port, calls PiDog movement methods, executes the supplied traceback, or connects to user hardware.

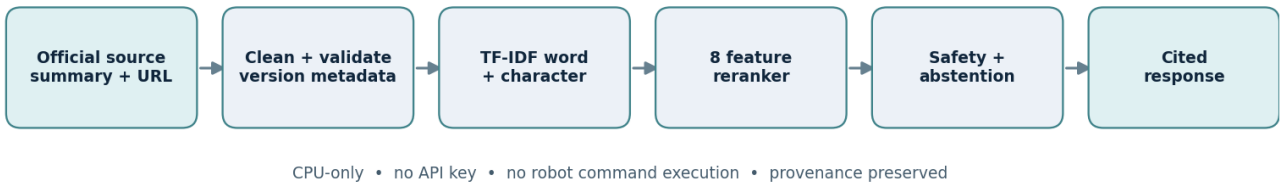


Figure 4. Retrieval and answer control flow (the local implementation does not use a generative LLM).

Answer decision contract

Condition	Behavior
Hazardous modification language	Escalate; do not provide bypass/rewiring instructions
Recognized clearly unrelated request	Out-of-scope answer, no spurious robot solution
Vague unresolved symptom	Ask for version, subsystem and exact error
No applicable evidence / weak matching	Escalate and request more specific context
Supported evidence found	Return selected manufacturer-attributed summaries, source IDs and URLs

Evidence-strength limitation: the visible numerical signal is an uncalibrated score built from top result similarity, entity overlap and rank margin. It is intentionally labeled evidence strength instead of implying a validated probability that the instructions are true or safe. No hallucination-rate benchmark was conducted.

07 Retrieval and decision evaluation

A manually authored 43-question set covers calibration, version compatibility, action scheduling, installation, sensors, I2C, audio, power, ambiguous prompts, non-PiDog topics and unsafe requests. The fixed split includes 35 development and 8 test examples. Retrieval scores exclude examples without gold source IDs, while status and intent scores use all examples. The corpus and questions both originate from the same documentation domain, limiting external validity.

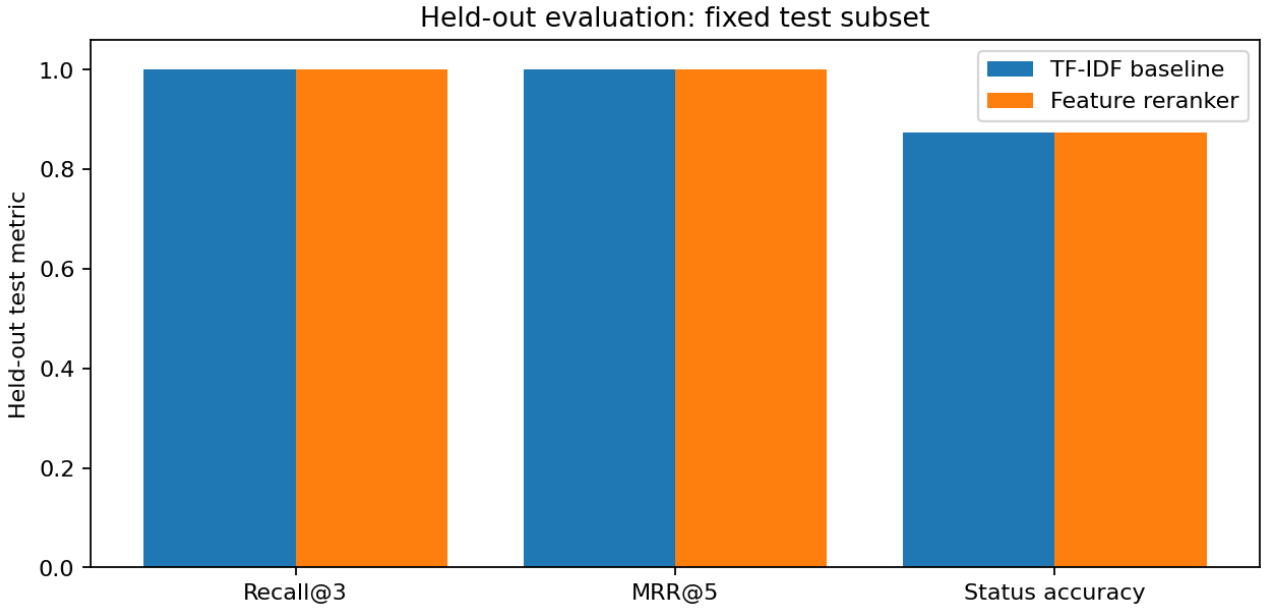


Figure 5. Eight-question fixed test slice: neither system has an advantage in retrieval here.

Metric	Baseline (dev)	Hybrid (dev)	Baseline (test)	Hybrid (test)
Recall@3	0.871	0.871	1.000	1.000
MRR@5	0.774	0.839	1.000	1.000
Status accuracy	0.943	0.971	0.875	0.875
Intent accuracy	0.857	0.857	0.750	0.750

Development results support the hypothesis that inexpensive engineered signals can improve early ranking. The eight-query test slice was inspected during prototype work, so it must be treated as exploratory rather than a genuinely untouched external test. There is no claim that the planned Recall@3 ≥0.85 target generalizes to real-world failures.

08 Representative assistant interactions

The following examples are literal records from `results/interactions.json` produced by calling the implemented code. Source citations are provided as links in the JSON and in the notebook. Text excerpts below are abbreviated to fit this report.

1. My PiDog wobbles after assembly

Outcome: answered | evidence strength: 0.133 | sources: zero_v1, zero_v2

1. For Standard/V1 PiDog, use the manufacturer-provided zeroing script before assembly: cd ~/pidog/examples ; sudo python3 servo_zeroing.py. Return the servos to zero before reassembling legs. This procedure differs from V2 zero button.

2. My V2 zero button works but Python servo control does not

Outcome: answered | evidence strength: 0.683 | sources: gpio_hat, zero_v2

1. If V2 Robot HAT zero button moves servos but Python calibration cannot, the Raspberry Pi to Robot HAT GPIO header connection may be faulty. SunFounder notes soldering issues in the Pi 40-pin header and Robot HAT socket; take clear pictures of both for manufacturer su...

3. How do I run ten push_up repetitions?

Outcome: answered | evidence strength: 0.387 | sources: actions, action_sequence

1. Pidog.do_action(action_name, step_count=1, speed=50) executes supported preset actions; documented names include sit, stand, forward, backward, turn_left, turn_right, trot, stretch, push_up, shake_head and wag_tail. step_count sets repetition count, speed controls ra...

4. The touch sensor has a HIGH reading

Outcome: answered | evidence strength: 0.640 | sources: touch, sensors

1. A LOW touch-sensor signal indicates touch. When the touch module does not respond, check cables, orientation and the relevant GPIO readout using a documented test. Do not assume HIGH means touched.

5. Ultrasonic says -1

Outcome: answered | evidence strength: 0.520 | sources: ultrasonic, sensors

1. For no ultrasonic reading or -1, SunFounder recommends cd ~/pidog/test && sudo python3 ultrasonic_test.py. Check that the white wire goes to GPIO17 and yellow wire to GPIO4; if -1 persists after verified wiring, contact support.

6. Can Standard PiDog use Raspberry Pi 5?

Outcome: answered | evidence strength: 0.303 | sources: versions, zero_v1, install_error

1. PiDog Standard/V1 supports Raspberry Pi 3B+, Raspberry Pi 4B and Zero 2W; it does not support Raspberry Pi 5. PiDog V2 has updated Robot HAT and servo circuitry and supports Raspberry Pi 3/4/5 and Zero 2W. Confirm your PiDog hardware version before following platform...

09 Safety, errors and failure analysis

Version filtering is mandatory before ranking. A Standard/V1 zeroing procedure is excluded for V2-specific prompts; a V2 zero-button instruction is excluded for Standard-specific prompts. The assistant cannot certify a hardware diagnosis. Camera advice explicitly prohibits live cable changes; servo calibration favors manufacturer instructions; battery-protection bypass requests are escalated.

Representative development and test failures

- Very short ambiguous prompts can still fail the heuristic classifier and be escalated rather than clarified.
- Intent-label rules can confuse sound/audio, RGB/I2C and hardware-version questions; ranking success and intent labeling are evaluated separately.
- Out-of-domain detection combines lexical heuristics with a similarity gate and is not robust against all adversarial prompts.
- High source relevance is not sufficient evidence that real-world instructions are electrically safe for an unknown robot model.
- When a source contains an error or an outdated instruction, extraction faithfully reproduces it; a source-review workflow is necessary.

The benchmark contains intentionally unsafe and out-of-scope prompts. The outcomes and failures are preserved per question in `results/per_question_results.csv` rather than silently cherry-picking successful demos.

What was not independently verified

Human groundedness rubric, citation correctness percent, real-world PiDog task-resolution rate, hardware calibration accuracy and error-reduction time have not been measured. These must remain marked as not evaluated; converting target thresholds from the midterm plan into “achieved” values would be misleading.

10 Engineering reproducibility and testing

The repository includes structured Python modules, a Colab notebook with executed outputs, a Gradio interface, pytest tests, source provenance, feature tables, visualizations, benchmark files and JSON/CSV evaluation artifacts. Environment requirements are provided in `pyproject.toml` and `requirements.txt`.

Task	Reproducible command
Install project and tests	python -m pip install -e '.[test,ui,online]'
Run test suite	python -m pytest -q
Evaluate and regenerate plots	python -m roboguide.evaluation
Ask a question	python -m roboguide "My PiDog wobbles"
Launch browser UI	python -m roboguide.app
Optional live docs acquisition	python -m roboguide.ingest --online

Observed execution in this sandbox: 19 pytest tests passed; the Colab-format notebook executed successfully using the local Python kernel; evaluation generated CSV, JSON, and four performance/data-quality figures. Internet-only data acquisition and external Gradio hosting were not executed here.

Dataset/version controls

- Rebuild the curated corpus from `data/make_corpus.py` and benchmark from `data/make_benchmark.py`.
- Keep source URL, fetch date, digest and human-reviewed version tag on expanded online snapshots.
- Never commit robot credentials, personal telemetry, private logs or API secrets.
- Freeze new evaluation questions before changing scoring thresholds; maintain a documented change log.
- Review any hardware procedure and source terms before publishing lengthy copied code.

11 Reflection and prioritized roadmap

The strongest outcome is a reproducible retrieval engine with transparent feature attribution, controlled version gating and extractive answers. The main trade-off is scale and semantic richness: a small manually curated corpus demonstrates engineering discipline but cannot prove broad coverage. The CPU-only design remains valuable because it works without cloud credits and fails safely when stronger generation is unavailable.

Priority	Milestone	Acceptance test
P0	Independent source ingestion; version/code review	80+ official pages/files with hashes, provenance, code-fidelity review
P0	External issue benchmark	150–250 labeled, non-template, held-out real questions
P1	MiniLM + FAISS comparison	CPU memory, Recall@3 and MRR compared to lexical baseline
P1	Calibrated abstention	Separate validation threshold + false-answer/false-refusal analysis
P1	Citation review	Blind check per adjacent claim and command against exact source
P2	Optional local LLM grounded synthesis	Answer supports only verified evidence; no invented API calls
P2	Real user study	Instructor-reviewed tasks on powered-down/safe robot setup and survey

The above milestones are proposals, not delivered capabilities. A full professional release should also add accessibility checks, software bill of materials and dependency scanning, licenses for source excerpts, model cards, incident logging without personal identifiers and a hard human-in-the-loop procedure for hazardous instructions.

12 Team collaboration and contribution verification

The course and instructor require an account of work already performed by each member, including who drafted the plan, diagrams and references—not only who was assigned upcoming tasks. The midterm lists the roster and planned ownership but does not establish each student's actual authored portions. Statements below are therefore proposed verification prompts; team members must revise them before submission.

José Tuozzo

Ownership assigned in the midterm: pipeline integration, evaluation framework, end-to-end QA and technical editing. Completed work to verify: describe 2–3 sentences covering actual implementation/code changes, authored report sections, midterm plan writing, diagrams, references, experiments and dates. Cite Git commits or a shared document revision history where available. Do not attribute the current code to this student unless verified.

Kim Nguyen

Ownership assigned in the midterm: source collection, manifest and version tagging, parsing, deduplication and quality review. Completed work to verify: describe 2–3 sentences covering actual implementation/code changes, authored report sections, midterm plan writing, diagrams, references, experiments and dates. Cite Git commits or a shared document revision history where available. Do not attribute the current code to this student unless verified.

Huu Huynh

Ownership assigned in the midterm: retrieval/index research, response/UI features, feature extraction and user testing. Completed work to verify: describe 2–3 sentences covering actual implementation/code changes, authored report sections, midterm plan writing, diagrams, references, experiments and dates. Cite Git commits or a shared document revision history where available. Do not attribute the current code to this student unless verified.

Draft provenance: This prototype/report was prepared with AI coding and editing assistance. José and teammates should inspect it, run it, independently verify sources, and correct the author and contribution statements before presenting it as group work. Academic responsibility remains with the submitting students.

Verification checklist	Team confirmation
Who drafted midterm executive, methodology and references?	[To be completed]
Who designed the three midterm architecture/schema/pipeline diagrams?	[To be completed]
Who wrote, reviewed, tested or corrected implementation modules?	[To be completed]
Who ran independent tests and checked numeric evidence?	[To be completed]
Agreement by José / Kim / Huu with dates	[To be completed]

13 References and submission inventory

[1] SunFounder. PiDog Documentation. https://docs.sunfounder.com/projects/pidog/en/latest/

[2] SunFounder. PiDog FAQ. https://docs.sunfounder.com/projects/pidog/en/latest/faq.html

[3] SunFounder. Calibrating PiDog. https://docs.sunfounder.com/projects/pidog/en/latest/python/py_0_calibrate.html

[4] SunFounder. Preset Action API. https://docs.sunfounder.com/projects/pidog/en/latest/python/py_b6_preset_action.html

[5] SunFounder. Robot HAT Documentation. https://docs.sunfounder.com/projects/robot-hat/en/latest/

[6] SunFounder. PiDog GitHub Repository. https://github.com/sunfounder/pidog

[7] scikit-learn. TF-IDF Vectorizer Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

[8] scikit-learn. Permutation Importance Documentation. https://scikit-learn.org/stable/modules/permutation_importance.html

Submission inventory

- This PDF: FN_JoseTuozzo_TungTungTungSahar_ITAI2377.pdf
- notebooks/RoboGuide_Colab.ipynb — complete, executed, well-commented notebook
- roboguide/ — reusable Python package, Gradio app, preprocessing and evaluation
- data/ — explicit source-attributed curated corpus and manually authored benchmark
- docs/figures/ — before/after quality, baseline, importance, corpus/architecture plots
- results/ — exact metrics.json, case-level results CSV, processed sample and interaction traces
- docs/CONTRIBUTIONS_TEMPLATE.md — sign-off required to finalize group authorship.

Important: The original assignment paste contains a contradictory old May 1, 2025 timestamp below a November 2026 Canvas due date. Use the current Canvas course due-date setting (December 2, 2026, 11:59 p.m.) as your reference, and verify with the instructor before final submission.